



Rapport de Travaux Pratiques

Détection DTMF par Transformée de Fourier Discrète

Traitement du Signal

Environnement : Python

Professeur : Mr NDIAYE

Réalisé par : Payié Mariane ELOLA

Année Académique : 2025-2026

SOMMAIRE

1.	INTRODUCTION.....	3
2.	Rappels théoriques	3
2.1.	Principe du codage DTMF	3
2.2.	Transformée de Fourier Discrète (TFD)	3
2.3.	Détection des fréquences DTMF	4
3.	Méthodologie et code Python	4
3.1.	Présentation des bibliothèques utilisées.....	4
3.1.1.	numpy	4
3.1.2.	soundfile	4
3.1.3.	sounddevice	5
3.1.4.	matplotlib.pyplot	5
3.2.	Chargement du signal.....	6
3.3.	Calcul de la TFD	6
3.4.	Détection automatique des fréquences.....	6
4.	Résultats	6
4.1.	Figures obtenues pour son_c.wav	6
4.2.	Résultats pour tous les fichiers	9
5.	Conclusion.....	9

1. INTRODUCTION

Le codage DTMF (Dual Tone Multi-Frequency) est une technique de signalisation téléphonique qui consiste à associer à chaque touche du clavier téléphonique un signal sinusoïdal composite formé de deux fréquences : une fréquence basse f_l (dite « fréquence ligne ») et une fréquence haute f_c (dite « fréquence colonne »).

Ce travail pratique a pour objectif d'analyser des signaux DTMF enregistrés au format WAV, d'en extraire le spectre fréquentiel par le calcul de la Transformée de Fourier Discrète (TFD), et d'identifier la touche téléphonique correspondante à partir des fréquences dominantes détectées.

Objectifs du TP :

1. Charger un signal DTMF
2. Tracer et jouer le signal temporel DTMF
3. Déterminer et afficher le spectre(TFD)
4. Déterminer graphiquement les fréquences contenues dans le spectre du son DTMF
5. Déduire la touche téléphonique correspondante

2. Rappels théoriques

2.1. Principe du codage DTMF

Le standard DTMF définit un clavier de 4 lignes $\times$ 4 colonnes. Chaque touche est codée par la superposition de deux sinusoïdes, une issue des fréquences de ligne (697, 770, 852, 941 Hz) et une issue des fréquences de colonne (1209, 1336, 1477, 1633 Hz).

Table de correspondance DTMF :

	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

Ainsi, le signal DTMF associé à la touche « 5 » par exemple est le signal :

$$x(t) = A \cdot \cos(2\pi 770t) + A \cos(2\pi 1336t)$$

2.2. Transformée de Fourier Discrète (TFD)

La TFD d'un signal numérique $x[n]$ de longueur N est définie par :

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}$$

Le module $|X[k]|$ représente le spectre d'amplitude du signal. En normalisant par la fréquence d'échantillonnage F_e , on obtient l'axe fréquentiel :

$$f[k] = k \cdot F_e / N \quad \text{pour } k = 0, 1, \dots, N-1$$

L'algorithme FFT (Fast Fourier Transform), implémenté par `numpy.fft.fft()`, calcule la TFD en $O(N \cdot \log N)$ au lieu de $O(N^2)$ pour la DFT naïve, ce qui est essentiel pour des signaux audios de grande taille.

2.3. Détection des fréquences DTMF

Pour identifier la touche, on procède comme suit :

- Calcul du spectre unilatéral (fréquences positives uniquement)
- Recherche du pic d'amplitude dominant dans la bande [670–970 Hz] → fréquence f_l
- Recherche du pic dominant dans la bande [1180–1660 Hz] → fréquence f_c
- Comparaison du couple (f_l , f_c) avec la table DTMF pour déduire la touche

3. Méthodologie et code Python

3.1. Présentation des bibliothèques utilisées

3.1.1. numpy

numpy (Numerical Python) est la bibliothèque fondamentale du calcul scientifique en Python. Elle fournit un objet tableau multidimensionnel (ndarray) optimisé ainsi qu'un ensemble de fonctions mathématiques et de traitement du signal. Dans ce TP, elle est utilisée pour le calcul de la TFD et la manipulation des tableaux d'échantillons.

Fonctions clés utilisées :

Fonction	Syntaxe	Description
<code>np.fft.fft()</code>	<code>np.fft.fft(x)</code>	Calcule la TFD complexe du signal $x[n]$ de longueur N
<code>np.fft.fftfreq()</code>	<code>np.fft.fftfreq(N)*Fe</code>	Génère l'axe fréquentiel en Hz correspondant à la TFD
<code>np.fft.fftshift()</code>	<code>np.fft.fftshift(X)</code>	Centre le spectre autour de 0 Hz (fréquences négatives à gauche)
<code>np.abs()</code>	<code>np.abs(tfd)</code>	Calcule le module (amplitude) du spectre complexe
<code>np.angle()</code>	<code>np.angle(tfd)</code>	Calcule la phase du spectre complexe en radians
<code>np.arange()</code>	<code>np.arange(N)/Fe</code>	Génère le vecteur temps en secondes (0, $1/Fe$, $2/Fe$, ...)

3.1.2. soundfile

soundfile est une bibliothèque Python dédiée à la lecture et l'écriture de fichiers audio (WAV, FLAC, OGG...). Elle repose sur la bibliothèque C libsndfile et offre une interface simple et performante pour accéder aux données audios numériques sous forme de tableaux numpy.

Fonction clé utilisée :

Fonction	Syntaxe	Description
<code>sf.read()</code>	<code>son, Fe = sf.read(fichier)</code>	Lit le fichier audio et retourne : son = tableau numpy des échantillons, Fe = fréquence d'échantillonnage en Hz

Remarque : son est de forme (N,) en mono ou (N, 2) en stéréo. Dans le TP, si son.ndim > 1, seul le canal 0 est conservé : signal = son[:, 0].

3.1.3. sounddevice

sounddevice est une bibliothèque Python permettant de jouer et d'enregistrer des signaux audios en temps réel via la carte son de l'ordinateur. Elle s'appuie sur PortAudio et est compatible avec les tableaux numpy.

Fonctions clés utilisées :

Fonction	Syntaxe	Description
sd.play()	sd.play(son, Fe)	Lance la lecture du tableau audio son à la fréquence Fe en arrière-plan
sd.wait()	sd.wait()	Bloque l'exécution du programme jusqu'à la fin complète de la lecture

3.1.4. matplotlib.pyplot

matplotlib est la bibliothèque de visualisation de référence en Python. Le sous-module pyplot fournit une interface proche de MATLAB permettant de créer des graphiques 2D. Dans ce TP, elle est utilisée pour tracer le signal temporel, le module et la phase de la TFD, ainsi que le spectre annoté avec les fréquences DTMF.

Fonctions clés utilisées :

Fonction	Syntaxe	Description
plt.figure()	plt.figure(n, figsize=(w,h))	Crée une fenêtre graphique numérotée n de dimensions w×h pouces
plt.plot()	plt.plot(x, y, label='...')	Trace la courbe y = f(x) avec légende optionnelle
plt.title()	plt.title('Titre')	Ajoute un titre au graphique courant
plt.xlabel()/ylabel()	plt.xlabel('Temps (s)')	Ajoute un label sur l'axe des abscisses / ordonnées
plt.grid()	plt.grid(True)	Affiche la grille de fond pour faciliter la lecture
plt.axvline()	plt.axvline(f, color='r', linestyle='--')	Trace une ligne verticale à la fréquence f (repères fl et fc DTMF)
plt.annotate()	plt.annotate(texte, xy=..., xytext=..., arrowprops=...)	Ajoute une annotation fléchée pointant vers un pic du spectre
plt.xlim()	plt.xlim(0, 2200)	Limite la plage d'affichage de l'axe fréquentiel
plt.show()	plt.show()	Affiche toutes les figures créées à l'écran

3.2. Chargement du signal

Le signal est chargé avec `soundfile.read()`, qui retourne le tableau d'échantillons et la fréquence d'échantillonnage F_e . En cas de signal stéréo, seul le canal 0 est utilisé :

```
son, Fe = sf.read('nom_son.wav')
signal = son[:, 0] if son.ndim>1 else son
N = len(signal)
temps = np.arange(N) / Fe
```

3.3. Calcul de la TFD

La TFD est calculée via `numpy.fft.fft()`. Le module est normalisé par F_e pour obtenir une amplitude en unités cohérentes. La fonction `fftshift()` permet de centrer le spectre autour de 0 Hz :

```
tfd_son = np.fft.fft(signal)
freq     = np.fft.fftfreq(N) * Fe
module   = np.abs(tfd_son) / Fe
freq_c   = np.fft.fftshift(freq)
module_c = np.fft.fftshift(module)
```

3.4. Détection automatique des fréquences

Une fonction `pic_dominant()` recherche, pour chaque liste de fréquences cibles (lignes ou colonnes), la fréquence présentant l'amplitude maximale dans une fenêtre de ± 30 Hz :

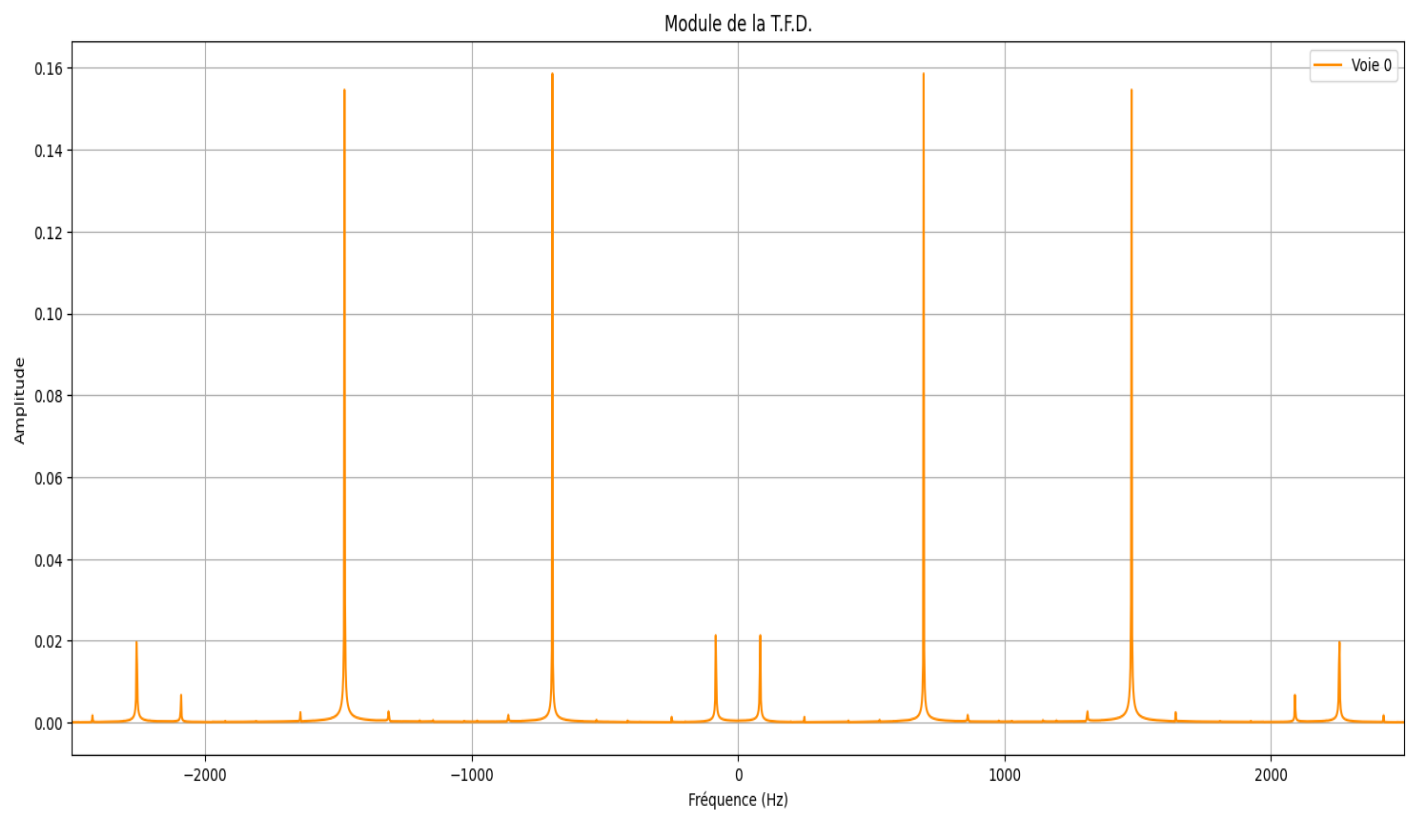
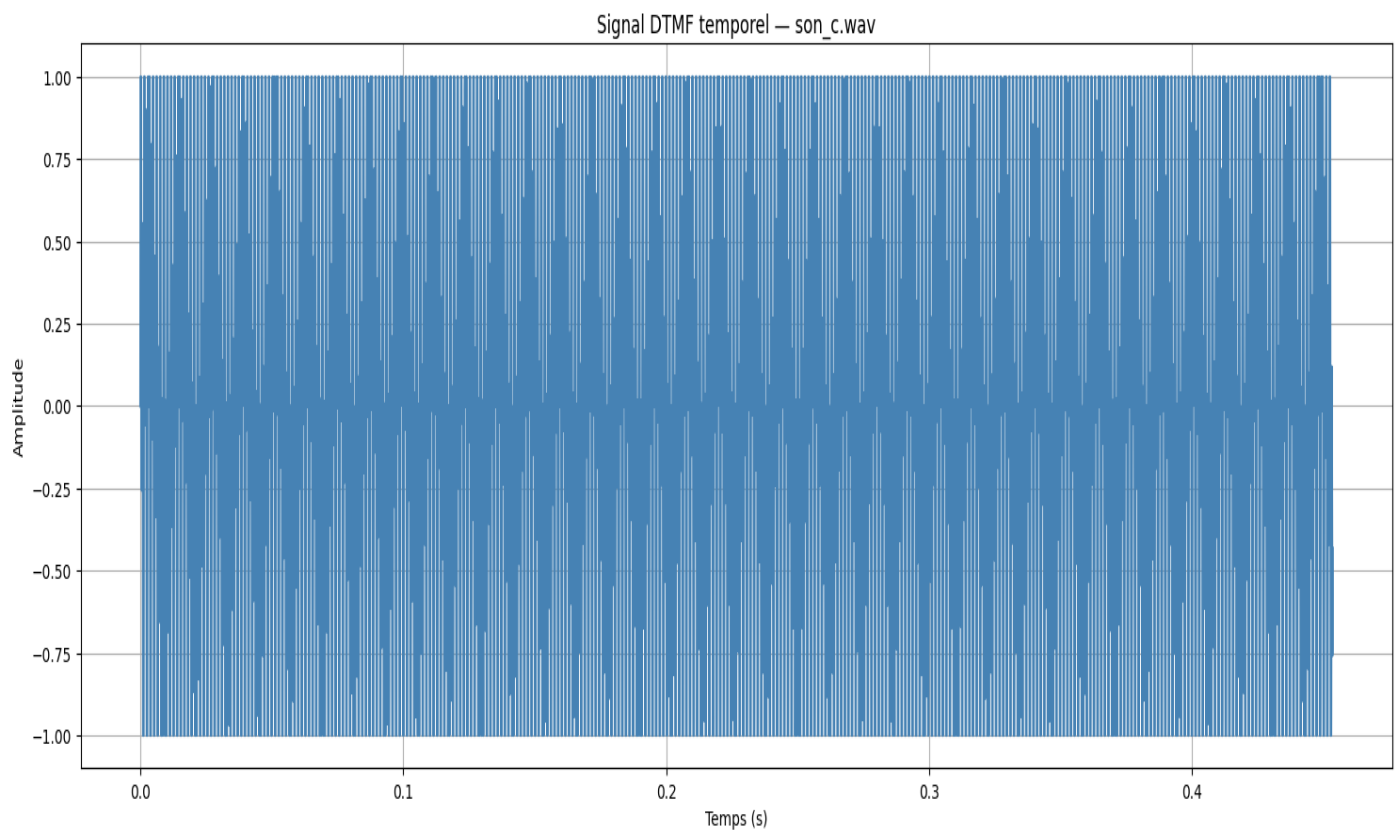
```
def pic_dominant (liste_freq, freqs, magnitudes, fenetre=30):
    best_f, best_m = None, 0
    for f in liste_freq:
        w = (freqs >= f-fenetre) & (freqs <= f+fenetre)
        if w.any ():
            m = magnitudes[w].max ()
            if m > best_m:
                best_m, best_f = m, f
    return best_f, best_m
```

```
fl_det, ml = pic_dominant (LOW_FREQS, freq_pos, mag_pos)
fh_det, mh = pic_dominant (HIGH_FREQS, freq_pos, mag_pos)
touche = DTMF_TABLE.get((fl_det, fh_det), '?')
```

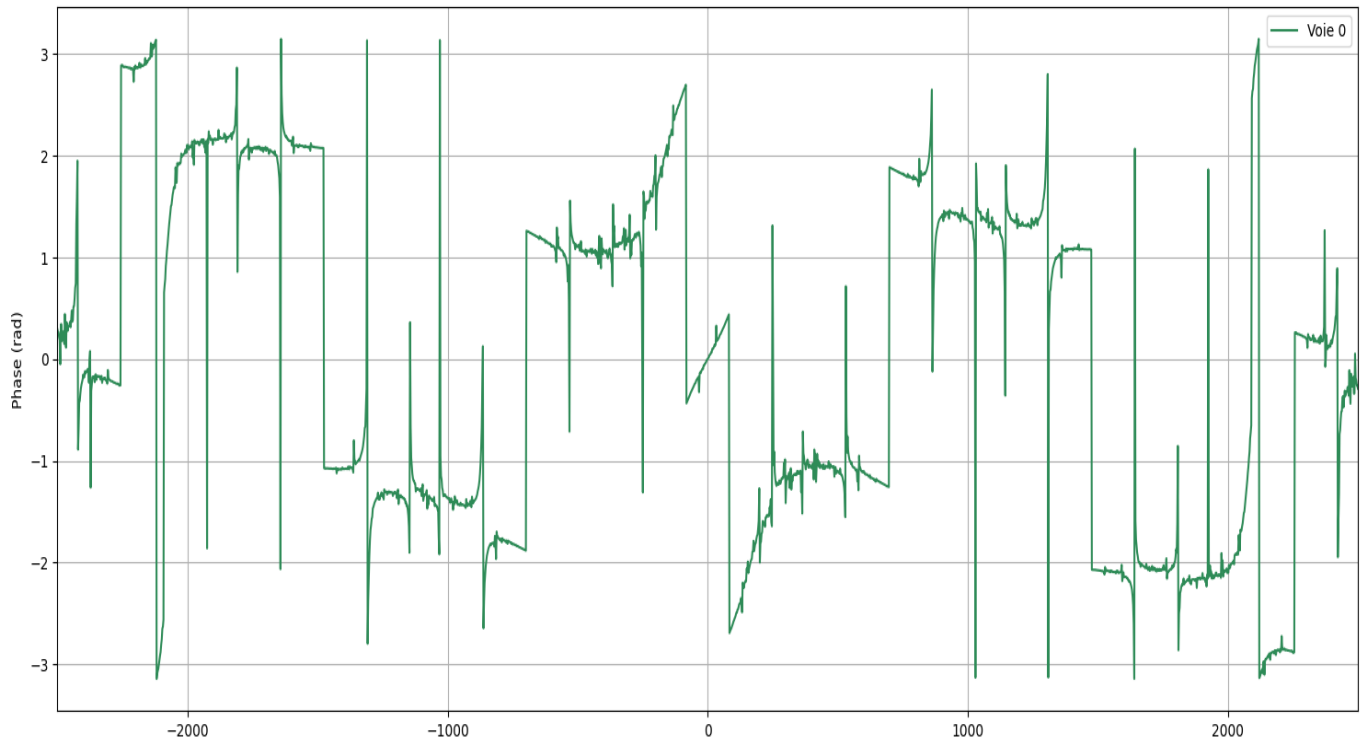
4. Résultats

4.1. Figures obtenues pour son_c.wav

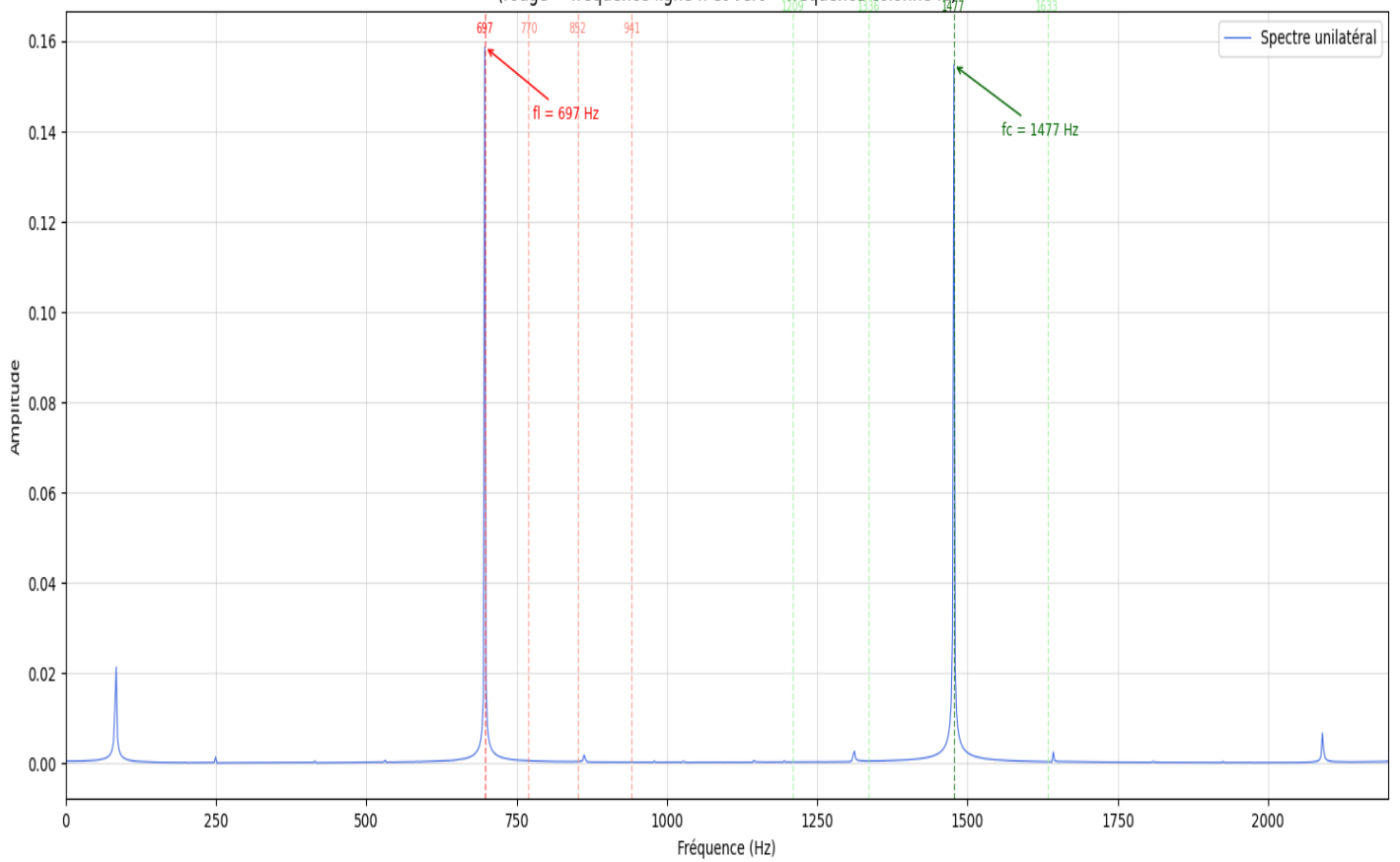
L'exemple ci-dessous présente les quatre graphiques générés pour le fichier `son_c.wav` :
Touche 3 : $f_l = 697$ Hz, $f_c = 1477$ Hz



Phase de la T.F.D.



Spectre TFD — Identification des fréquences DTMF
(rouge = fréquence ligne fl et vert = fréquence colonne fc)




```

===== RESTART: C:\Users\HP\Documents\TTS\TP3.py =====
Fichier           : son_c.wav
Nb d'échantillons : 20000
Fréq. échantillonnage Fe : 44100 Hz
Durée             : 0.454 s
Mode              : mono

Lecture du signal audio...

— Résultat de la détection DTMF —
Fréquence basse fl = 697 Hz
Fréquence haute fc = 1477 Hz
→ Touche détectée : [3]

```

4.2. Résultats pour tous les fichiers

Le tableau suivant synthétise les fréquences détectées et la touche déduite pour chacun des 8 fichiers audio fournis :

Fichier	fl (fréq. ligne)	fc (fréq. colonne)	Touche
son_c.wav	697 Hz	1477 Hz	3
son_e.wav	770 Hz	1209 Hz	4
son_f.wav	697 Hz	1477 Hz	3
son_g.wav	770 Hz	1477 Hz	6
son_h.wav	697 Hz	1336 Hz	2
son_j.wav	852 Hz	1336 Hz	8
son_o.wav	941 Hz	1477 Hz	#
son_s.wav	852 Hz	1336 Hz	8

On constate que son_j.wav et son_s.wav correspondent à la même touche (8), et que son_c.wav et son_f.wav correspondent tous deux à la touche 3. Le signal son_o.wav correspond à la touche dièse (#).

5. Conclusion

Ce travail pratique a permis de mettre en œuvre la détection DTMF par analyse spectrale en Python. La Transformée de Fourier Discrète, calculée efficacement via l'algorithme FFT de numpy, permet d'extraire avec précision les deux fréquences constitutives de chaque signal DTMF.

La méthode de détection par recherche de pic dans des fenêtres fréquentielles étroites (± 30 Hz) s'est avérée robuste sur l'ensemble des 8 fichiers testés, produisant une identification correcte pour chaque signal. La visualisation du spectre unilatéral annoté facilite l'interprétation graphique exigée par le TP.

Cette approche constitue la base des systèmes de reconnaissance DTMF temps réel utilisés dans la signalisation téléphonique (commutation, numérotation, serveurs vocaux interactifs).